

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

简单动态规划：从一维到二维

Introduction to Dynamic Programming

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 14 日

目录

1. 动态规划入门
2. 一维 DP 经典题
3. 二维 DP 经典题
4. 总结



目录

1 动态规划入门

2 一维 DP 经典题

3 二维 DP 经典题

4 总结

什么是动态规划

动态规划（Dynamic Programming, DP）：把一个大问题拆成若干个**相互重叠**的子问题，先解决子问题并**保存结果**，再用子问题的答案拼出原问题的答案。

- **最优子结构**：大问题的最优解，由子问题的最优解拼成
- **重叠子问题**：不同分支反复用到同一个子问题
- **无后效性**：当前状态确定后，后续决策只与当前状态有关

$$\text{DP} = \text{状态定义} + \text{转移方程} + \text{边界条件}$$

DP 和递归的区别

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

朴素递归

- 从原问题往下拆
- 同一个子问题被算很多次
- 复杂度常常是指数级
- 例：斐波那契 $O(2^n)$

动态规划

- 从最小子问题往上推
- 每个子问题只算一次
- 结果存进数组，随时复用
- 斐波那契优化到 $O(n)$

DP 的本质：用空间换时间，把重复计算变成查表

DP 的解题四步法

1. **定义状态**：$dp[\ldots]$ 表示什么？这是最关键的一步
2. **找转移方程**：$dp[i]$ 由哪些更小的状态推出
3. **确定边界**：最小规模时 dp 的值是多少
4. **确定顺序**：从小规模往大规模推，保证用到的状态已经算好

状态定义错了，后面全错；状态定义对了，方程往往一目了然

一维 DP 和二维 DP

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

一维 DP

- 状态只用一个下标
- 形式：$dp[i]$
- 典型：爬楼梯、打家劫舍
- 答案往往就是 $dp[n]$

二维 DP

- 状态需要两个下标
- 形式：$dp[i][j]$
- 典型：背包、最长公共子序列
- 答案在 $dp[n][m]$ 或某一行/列的最值

先从一维入门，理解套路后再上二维

目录

1 动态规划入门

2 一维 DP 经典题

3 二维 DP 经典题

4 总结

一维 DP 之一：爬楼梯

问题：有 n 级台阶，每次可以走 1 级或 2 级，问走到第 n 级共有多少种走法。

四步法

- 状态：$dp[i]$ 表示走到第 i 级的走法数
- 转移：最后一步只能是走 1 级或走 2 级，$dp[i] = dp[i-1] + dp[i-2]$
- 边界：$dp[1] = 1$，$dp[2] = 2$
- 顺序：i 从 3 到 n 依次递推

当前的走法数，等于“从上一级走一步来”与“从上两级走两步来”之和

代码：爬楼梯

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
// dp[i] 表示走到第 i 级的走法数
int climbStairs(int n) {
    if (n <= 2) return n;
    vector<int> dp(n + 1, 0);
    dp[1] = 1;
    dp[2] = 2;
    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];   // 转移方程 *
    }
    return dp[n];
}

// 空间优化：只需保留前两级
int climbStairsOpt(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;                // a = dp[i-2], b = dp[i-1]
    for (int i = 3; i <= n; i++) {
        int c = a + b;
```

代码：爬楼梯


```
19            a = b;
20            b = c;
21        }
22        return b;
23    }
```

答案只依赖前两项，可以用滚动变量把空间降到 $O(1)$。

一维 DP 之二：打家劫舍

问题：一条街上有 n 户人家，第 i 户有 $a[i]$ 元。不能偷相邻的两户，问最多能偷多少钱。

四步法

- 状态：$dp[i]$ 表示前 i 户能偷到的最大金额
- 转移：第 i 户要么不偷（继承 $dp[i-1]$），要么偷（$dp[i-2]+a[i]$）
- 方程：$dp[i] = \max(dp[i-1],\ dp[i-2]+a[i])$
- 边界：$dp[0]=0$，$dp[1]=a[1]$

每一步只有“偷”和“不偷”两种选择，取较大者

代码：打家劫舍


```cpp
// dp[i] 表示前 i 户能偷到的最大金额
int rob(vector<int>& a) {
    int n = a.size();
    if (n == 0) return 0;
    if (n == 1) return a[0];
    vector<int> dp(n + 1, 0);
    dp[0] = 0;
    dp[1] = a[0];
    for (int i = 2; i <= n; i++) {
        // 不偷第 i 户，或偷第 i 户并加上前 i-2 户的最优解 *
        dp[i] = max(dp[i-1], dp[i-2] + a[i-1]);
    }
    return dp[n];
}

// 空间优化：只保留前两个状态
int robOpt(vector<int>& a) {
    int prev = 0, cur = 0;
```

代码：打家劫舍

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19      for (int x : a) {
20          int next = max(cur, prev + x);
21          prev = cur;
22          cur = next;
23      }
24      return cur;
25  }
```

一维 DP 之三：最大子数组和

问题：给定数组 $a[1..n]$，找一个连续子数组，使其元素之和最大。

四步法

- 状态：$dp[i]$ 表示以 $a[i]$ **结尾**的最大子数组和
- 转移：要么接在前一个后面，要么从 $a[i]$ 重新开始
- 方程：$dp[i] = \max(dp[i-1] + a[i],\ a[i])$
- 答案：$\max_i dp[i]$，而不是 $dp[n]$

状态必须“以 i 结尾”，答案才需要再取一次最大值

代码：最大子数组和


```cpp
// dp[i] 表示以 a[i] 结尾的最大子数组和
int maxSubArray(vector<int>& a) {
    int n = a.size();
    vector<int> dp(n, 0);
    dp[0] = a[0];
    int ans = dp[0];
    for (int i = 1; i < n; i++) {
        dp[i] = max(dp[i-1] + a[i], a[i]);   // 接上或重开 *
        ans = max(ans, dp[i]);
    }
    return ans;
}

// 空间优化：滚动变量
int maxSubArrayOpt(vector<int>& a) {
    int cur = a[0], ans = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        cur = max(cur + a[i], a[i]);
```

代码：最大子数组和


```
19          ans = max(ans, cur);
20      }
21      return ans;
22  }
```

一维 DP 小结

共同套路

- 状态通常是“前 i 个”或“以 i 结尾”
- 转移只依赖前 1 到 2 个状态
- 常数项可以从状态里直接读出来

常见变体

- 爬楼梯：$dp[i] = dp[i-1] + dp[i-2]$
- 打家劫舍：$\max$ 或加
- 最大子数组：$\max$ 累加或重开

一维 DP 的关键：想清楚 $dp[i]$ 到底表示什么

目录

1 动态规划入门

2 一维 DP 经典题

3 二维 DP 经典题

4 总结

二维 DP 之一：网格路径数

问题：$n \times m$ 的网格，从左上角走到右下角，每次只能向右或向下走一步，问有多少种走法。

四步法

- 状态：$dp[i][j]$ 表示走到 (i, j) 的路径数
- 转移：只能从上方或左方过来，$dp[i][j] = dp[i-1][j] + dp[i][j-1]$
- 边界：第一行、第一列全为 1
- 顺序：按行从上到下，每行从左到右

当前格子的方案数 = 上方格子方案数 + 左方格子方案数

代码：网格路径数


```cpp
// dp[i][j] 表示走到 (i, j) 的路径数
int uniquePaths(int n, int m) {
    vector<vector<int>> dp(n, vector<int>(m, 0));

    // 边界：第一行、第一列都只有一种走法
    for (int i = 0; i < n; i++) dp[i][0] = 1;
    for (int j = 0; j < m; j++) dp[0][j] = 1;

    for (int i = 1; i < n; i++)
        for (int j = 1; j < m; j++)
            dp[i][j] = dp[i-1][j] + dp[i][j-1];   // 从上方或左方来 *

    return dp[n-1][m-1];
}
```

带障碍的变体：障碍格子的 dp 直接置 0，其余转移不变。

二维 DP 之二：0/1 背包

问题：有 n 件物品，第 i 件重 $w[i]$、价值 $v[i]$，背包容量 W。每件物品只能取一次，求最大总价值。

四步法

- 状态：$dp[i][j]$ 表示前 i 件物品、容量 j 时的最大价值
- 转移：第 i 件要么不取，要么取（前提是 $j \geq w[i]$）
- 方程：$dp[i][j] = \max(dp[i-1][j],\ dp[i-1][j-w[i]] + v[i])$
- 边界：$dp[0][j] = 0$，任何一件都不选时价值为 0

每个物品只有“取”和“不取”两种选择，取的时候要先腾出容量

代码：0/1 背包（二维）


```cpp
// dp[i][j] 表示前 i 件物品，容量为 j 时的最大价值
int knapsack(int n, int W, vector<int>& w, vector<int>& v) {
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= W; j++) {
            dp[i][j] = dp[i-1][j];              // 不取第 i 件
            if (j >= w[i-1]) {
                // 取第 i 件，用前 i-1 件、容量 j - w[i-1] 的最优解 + 当前价值 *
                dp[i][j] = max(dp[i][j],
                               dp[i-1][j - w[i-1]] + v[i-1]);
            }
        }
    }
    return dp[n][W];
}
```

二维写法直观，是理解 0/1 背包的第一选择。

代码：0/1 背包（一维优化）


```cpp
// 一维优化：dp[j] 表示容量为 j 时的最大价值
int knapsackOpt(int n, int W, vector<int>& w, vector<int>& v) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        // 容量必须从大到小遍历，保证每件物品只被取一次 *
        for (int j = W; j >= w[i]; j--) {
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
        }
    }
    return dp[W];
}
```

一维写法必须倒序遍历容量；正序会变成“完全背包”，每个物品能取无限次。

二维 DP 之三：最长公共子序列

问题：给定两个字符串 A 和 B，求它们最长的公共子序列长度（子序列不要求连续）。

四步法

- 状态：$dp[i][j]$ 表示 A 的前 i 个字符与 B 的前 j 个字符的 LCS 长度
- 转移：若 $A[i] = B[j]$，则 $dp[i][j] = dp[i-1][j-1] + 1$
- 否则：$dp[i][j] = \max(dp[i-1][j],\ dp[i][j-1])$
- 边界：$dp[0][j] = dp[i][0] = 0$

字符相同就“对齐加一”；不同就让较长的一边继续延伸

代码：最长公共子序列


```cpp
// dp[i][j] 表示 A 前 i 个字符与 B 前 j 个字符的 LCS 长度
int lcs(string& A, string& B) {
    int n = A.size(), m = B.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (A[i-1] == B[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;   // 字符相同，对齐加一 *
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);   // 否则取较长一边 *
        }
    }
    return dp[n][m];
}
```

答案在 $dp[n][m]$，正好是两串全长的 LCS 长度。

二维 DP 小结

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

共同套路

- 状态由两个维度共同确定
- 转移依赖左、上、左上三个方向
- 边界通常是第一行和第一列

常见变体

- 网格路径：加法
- 0/1 背包：取 max
- 最长公共子序列：判断加一或取 max

画一张 dp 表格，把“每一步从哪里来”标出来，转移方程就清楚了

目录

1 动态规划入门

2 一维 DP 经典题

3 二维 DP 经典题

4 总结



DP 解题四步法回顾

1. **定义状态**：$dp[\ldots]$ 到底代表什么？
2. **转移方程**：当前状态由哪些前驱状态推出？
3. **边界条件**：最小规模时，dp 的值是多少？
4. **递推顺序**：保证用到的状态都已经被算过

状态定义是 DP 的灵魂；写方程之前，先在心里把“表格”画出来

经典题型速查

题型	状态	转移
爬楼梯	$dp[i]$：走到第 i 级	$dp[i-1]+dp[i-2]$
打家劫舍	$dp[i]$：前 i 户最优	$\max(dp[i-1], dp[i-2]+a[i])$
最大子数组	$dp[i]$：以 i 结尾	$\max(dp[i-1]+a[i], a[i])$
网格路径	$dp[i][j]$：走到 (i,j)	上 + 左
0/1 背包	$dp[i][j]$：前 i 件容量 j	$\max$(不取，取)
最长公共子序列	$dp[i][j]$：两串前 i,j	相等加一，否则取 max

先把这六道题的代码敲熟，再去看更复杂的变形

下一步学什么

- 掌握 DP 的四步法，能用状态定义推导转移方程
- 熟练写出六道入门题的代码，并会做空间优化
- 为后续学习更复杂的 DP 打基础：
  - 区间 DP、树形 DP、状态压缩 DP
  - 完全背包、多重背包、分组背包
  - 数位 DP、概率 DP

入门 DP 不在难题，而在把“状态定义”变成习惯

附录：常见错误

- 状态定义含糊：写方程时不知道 $dp[i]$ 到底代表什么
- 边界忘记初始化：$dp[0]$、第一行、第一列没写
- 递推顺序错误：用到了还没算好的状态
- 数组越界：下标从 1 开始时数组要多开一格
- $0/1$ 背包一维优化写成正序：变成完全背包，答案错误
- 最大子数组忘记对所有 $dp[i]$ 取最大值

附录：空间优化速查

场景	优化方式	注意点
只依赖 $dp[i-1]$ 与 $dp[i-2]$	滚动变量	先更新旧值再更新新值
只依赖上一行	一维数组	0/1 背包必须倒序
依赖 $dp[i-1][j-1]$ 与 $dp[i-1][j]$	一维数组 + 临时变量	提前把被覆盖的值存起来
状态转移方向单一	原地更新	确认覆盖后不再需要旧值

空间优化是锦上添花；先写对，再优化。